Name: **Ruiqi Liu**
Edge Impulse Link: https://studio.edgeimpulse.com/public/996805/live

Part 1

1. Raw inputs: State the raw inputs used in the Edge Impulse project.

The raw inputs are the acceleration in the x, y, and z directions.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

The type of features passed into the NN classifier are features extracted from the accX, accY, and accZ data. For example, frequency, RMS, maximum of each of these. There are 33 input features. Five of the actual feature names are: accX RMS, accX peak 1 height, accX peak 3 height, accX peak 1 freq, accY RMS.

3. NN classifier outputs: State the outputs of the NN classifier.

There are 2 outputs, nominal and off, which are the state of the fan that the classifier is trying to predict.

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

The anomaly detector uses 4 features. These features are selected from the set of 33 features used by the NN. The features are: accX RMS, accX peak 1 height, accY spectral power 2 - 5 Hz, accZ spectral power 2 - 5 Hz.

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano 33 BLE Sense and producing inference results.

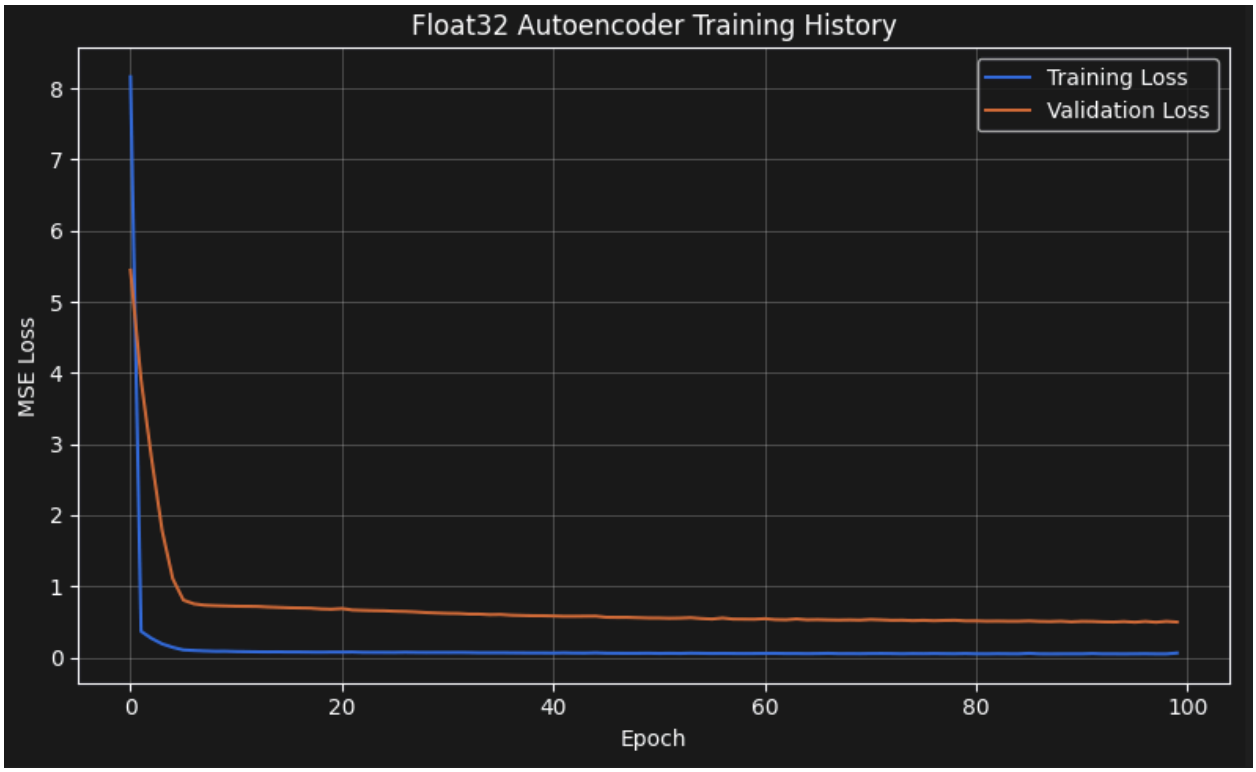
```
Flashed your Arduino Nano 33 BLE development board.
To set up your development with Edge Impulse, run 'edge-impulse-daemon'
To run your impulse on your development board, run 'edge-impulse-run-impulse'
whale@whale-fedora:~/shared/ee446/lab6$ edge-impulse-run-impulse
Edge Impulse impulse runner v1.38.0
[SER] Connecting to /dev/ttyACM0
[SER] Serial is connected, trying to read config...
Failed to parse snapshot line [  ]
[SER] Retrieved configuration
[SER] Device is running AT command version 1.8.0
[SER] Started inferencing, press CTRL+C to stop...
LSE
Inferencing settings:
        Interval: 10.00 ms.
        Frame size: 600
        Sample length: 2000 ms.
        No. of classes: 2
Starting inferencing, press 'b' to break
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 581 us, anomaly 6 ms, postprocessing 67 us
#Classification predictions:
  nominal: 0.843750
  off: 0.156250
Anomaly prediction: 220.692413
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 584 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.937500
  off: 0.062500
Anomaly prediction: 8.026061
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 584 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
```

6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

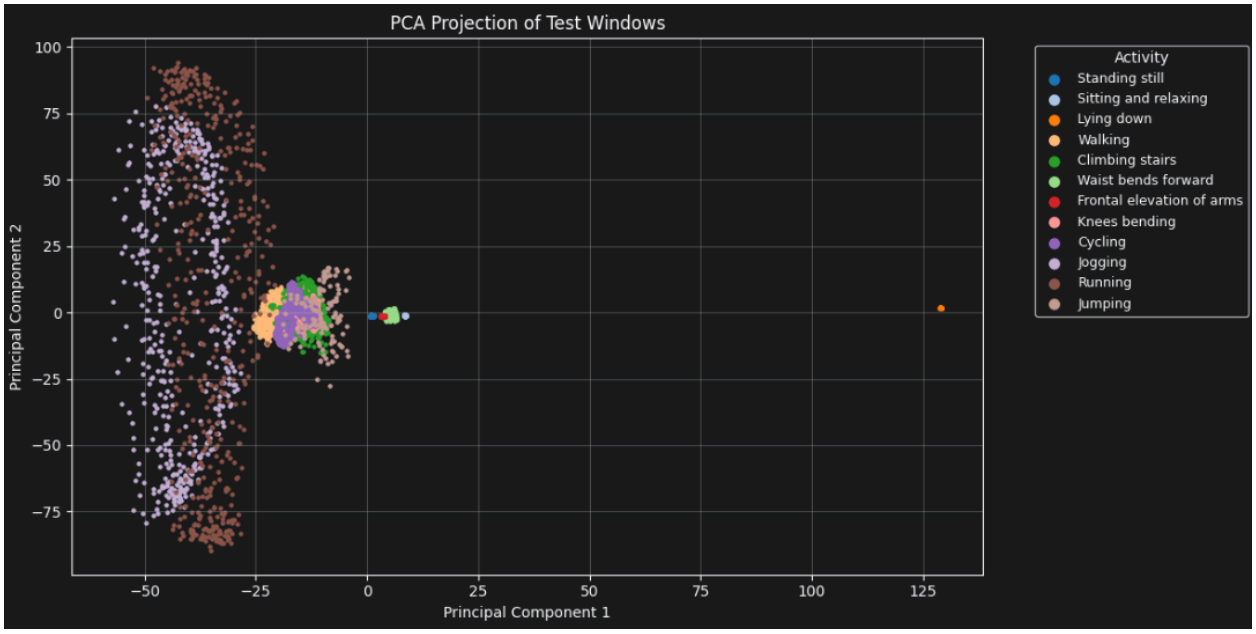
I observed that the anomaly prediction value can change greatly between inferences. The classifier output should not always be trusted. Even though its accuracy is very high, there may be outside interference that causes an anomaly or for the nominal/off prediction. However, the classifier can be trusted if there is not such outside interference.

Part 2

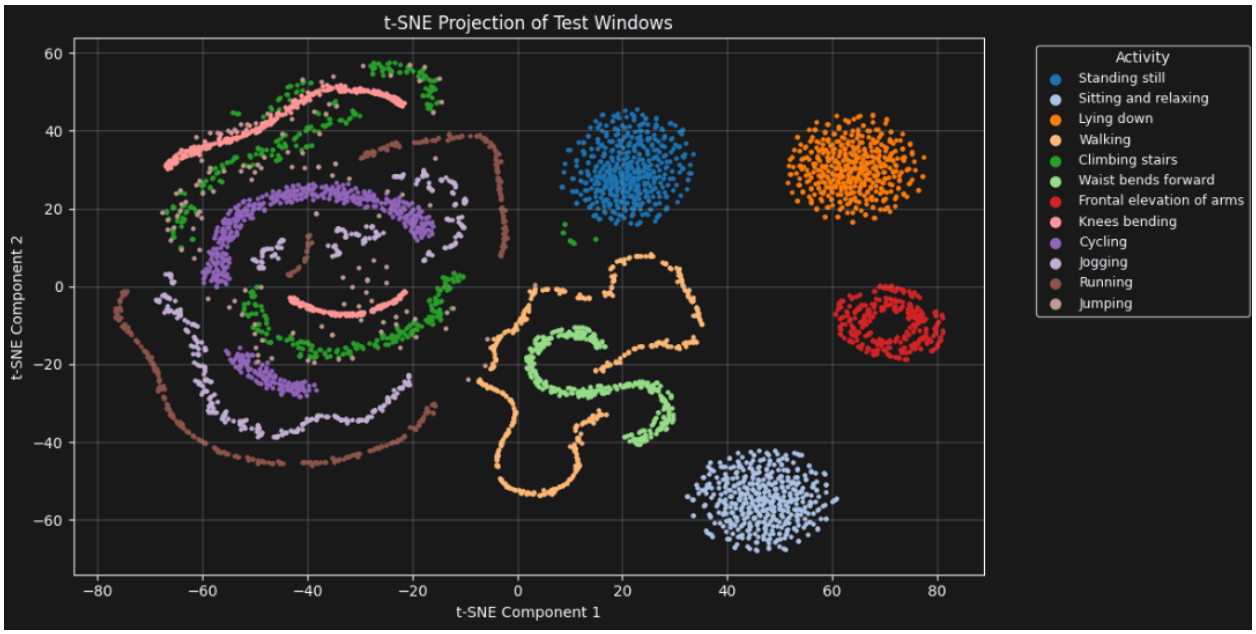
Training and validation loss curves



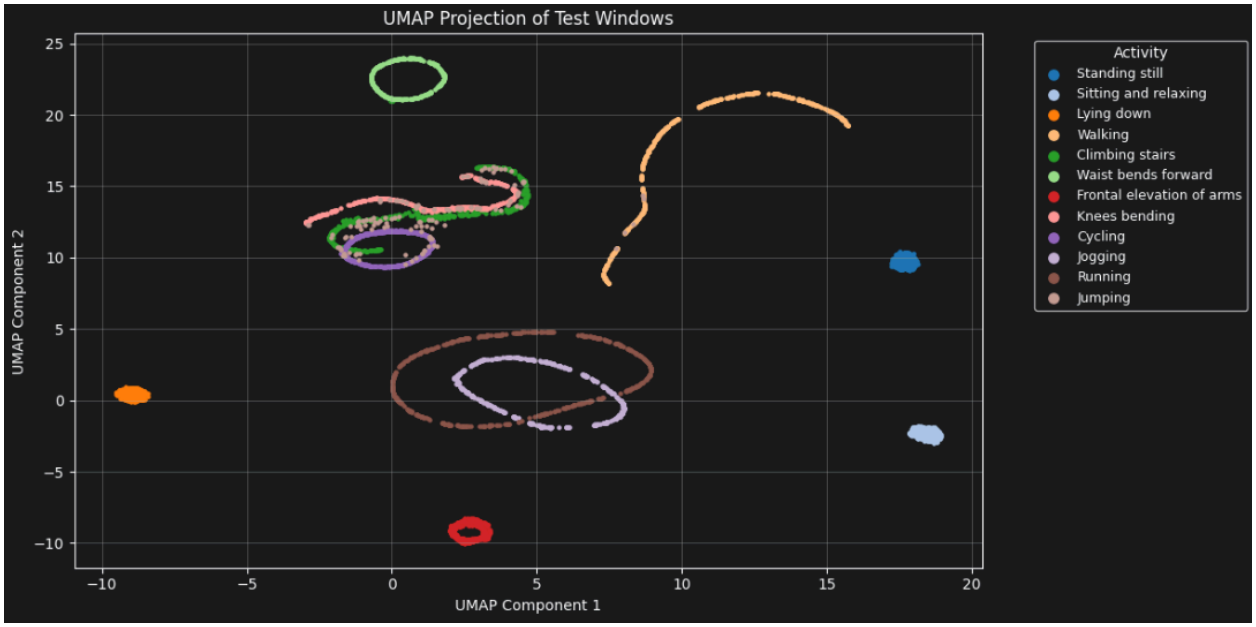
PCA visualization



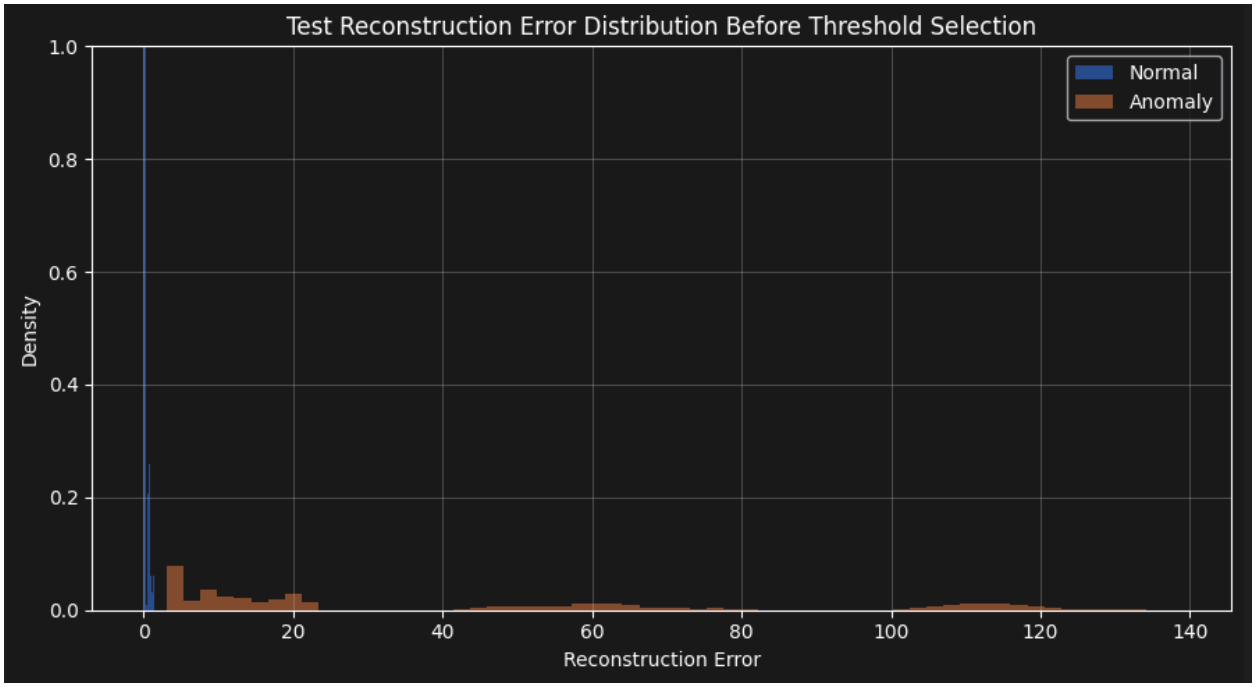
t-SNE visualization



UMAP visualization



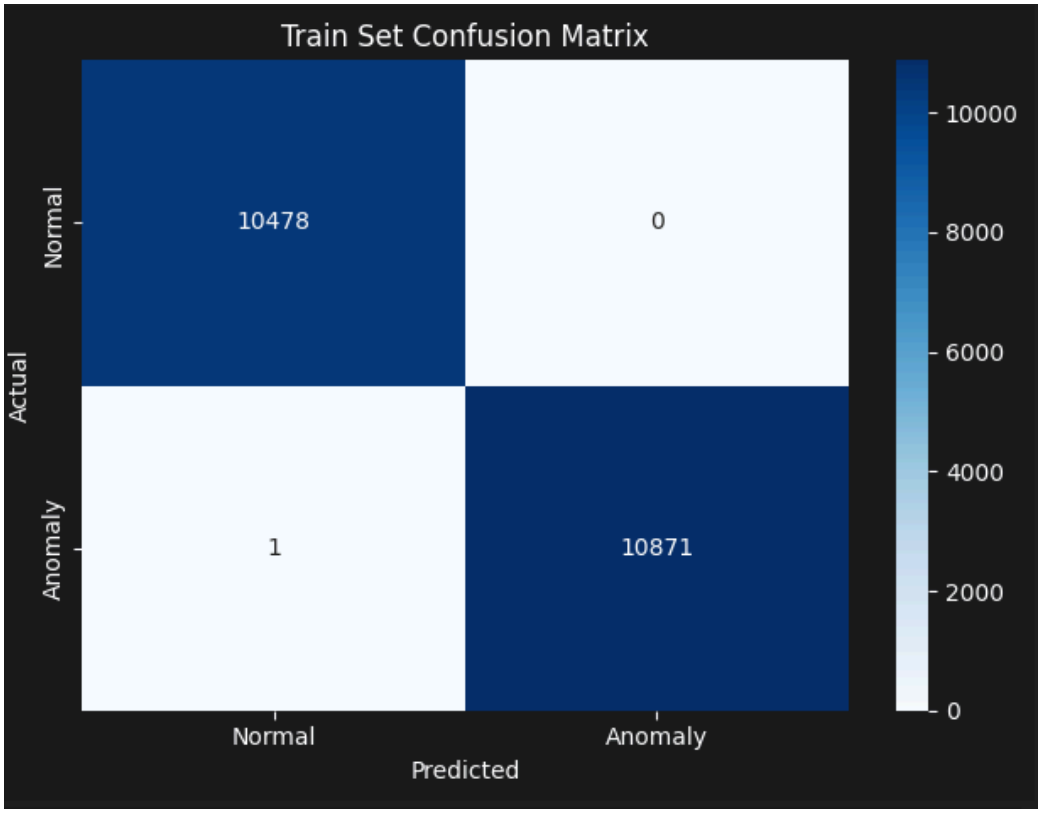
Reconstruction error distribution

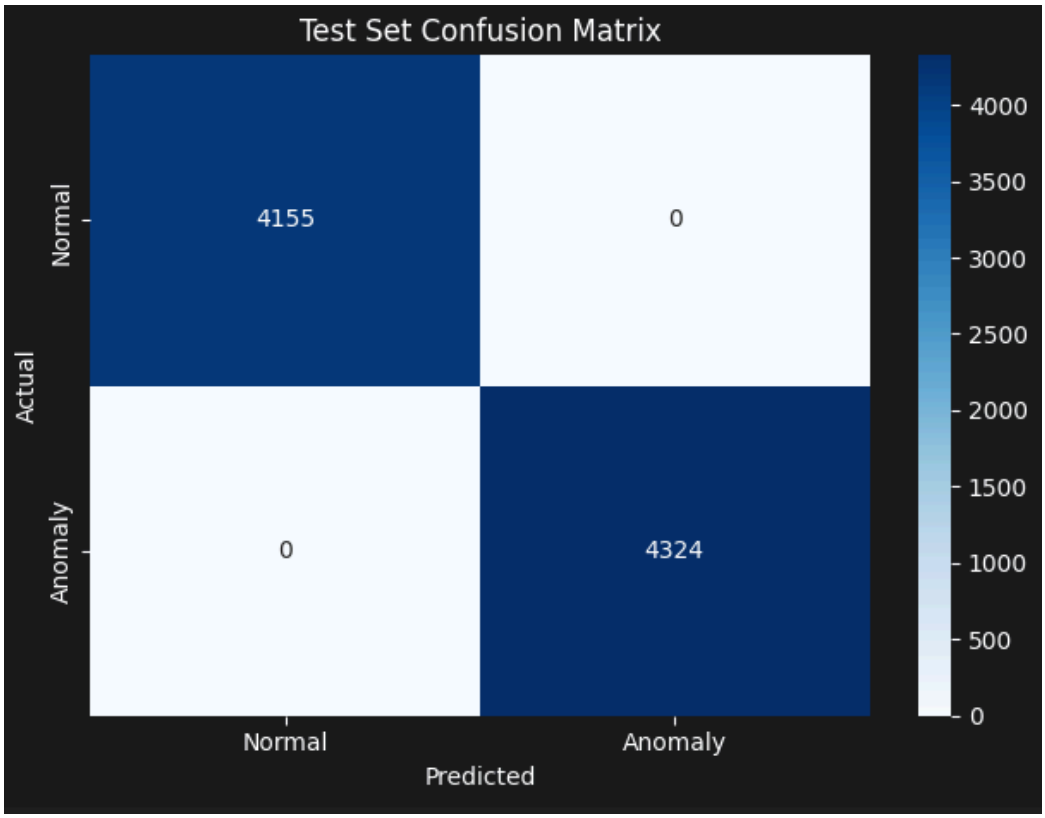


Selected anomaly threshold

My selected anomaly threshold is 2.5. I selected this based on the graph shown.

Confusion matrix





Classification report or evaluation metrics

```
Train set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350
```

```
Test set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```

Arduino serial monitor output showing reconstruction error and normal/anomaly detection results

```
Result: Normal activity
Reconstruction error: 1.874794
Result: Normal activity
Reconstruction error: 1.934064
Result: Normal activity
Reconstruction error: 1.924846
Result: Normal activity
Reconstruction error: 2.006826
Result: Normal activity
Reconstruction error: 2.108424
Result: Normal activity
Reconstruction error: 2.161922
Result: Normal activity
Reconstruction error: 2.355043
Result: Normal activity
Reconstruction error: 2.442163
Result: Normal activity
Reconstruction error: 2.673635
Result: Anomaly detected
Reconstruction error: 2.779637
Result: Anomaly detected
Reconstruction error: 2.974124
Result: Anomaly detected
Reconstruction error: 3.120611
Result: Anomaly detected
Reconstruction error: 3.225976
Result: Anomaly detected
Reconstruction error: 3.189147
Result: Anomaly detected
Reconstruction error: 3.318588
Result: Anomaly detected
Reconstruction error: 3.414471
```

What threshold did you choose for anomaly detection, and why?

My threshold chosen was 2.5. I chose this threshold because it gives almost a 100% accuracy, and I saw that it was around that value on the graph of blue vs orange.

How does reconstruction error help distinguish normal and anomalous activity?

Reconstruction error distinguishes normal and anomalous activity because the model is trained on the normal data, so the amount of error can determine if it is normal or anomalous activity.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The Arduino code needs the threshold value used in the python notebook. It also needs the autoencoder_model.cc file generated by the notebook.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The Arduino sketch uses the same model as the notebook, so it must use the same preprocessing assumptions in order for the data to be interpreted correctly.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

This anomaly detection method requires the microcontroller to always be on and do a lot of calculations, which can use a lot of power. Also, the quantization process might mess with the reconstruction error, which would affect the anomaly detection. If the microcontroller is not powerful enough for a more complex model, it might cause lag or inaccuracies.